Exercice 2
Advanced Computer Architecture

Anton Schelfhout, Yari Vanherpe

21 februari 2024



Instructor: A. Marinšek

1 Introduction

In this lab session we looked for the largest element in an array. Conventionally this is done in a simple for loop using the CPU without the use of threads. This report describes an two other approaches using the GPU and different threads, and compares their speed.

2 Method

During the lab session we made three methods, one using the CPU and two using the GPU. In this section we will shortly cover the methods and explain our reasoning.

2.1 A nested for loop on the CPU

This is the most basic function, just a simple for loop that compares the element at index i with the largest element found so far. If the element at index i is larger, then save it.

2.2 A GPU kernel employing atomic operation

This function uses the same principle only using multiple threads, each thread compares one element with the max value, and if it is larger replaces the max value with its own.
It is paramount that the compare and eventual write operation are atomic, meaning that no other thread can read the max value when another thread is using it. The max value thus can be seen as a critical section. This is achieved by using the ätomicMax"function, this function compares the two parameters, and stores the largest on the address of the first parameter. This is all done atomically.

2.3 A GPU kernel using reduction

The last method uses reduction to find the largest number. Instead of comparing each element to a shared max-value, it compares two values in the array and stores the largest at the lowest index.
To know which elements to compare, two variables are defined: the startIndex and step variables. The startIndex is initialized at 2*index for each thread, and the step variable at one. After each iteration we multiply these variables with two.
By doing this in a for loop we find the maximum. For instance thread 2 will start with the comparison if element 4 and 5, next iteration it will compare element 8 and 10, and so on. at the end of the for loop we will find the largest number at index 0 in the array.
When running this algorithm it the array needs a size of a power of two, this can be done by either appending zeros to the end of the array, or by finding the largest power of two that is smaller than size and extracting the largest element of the sub-array from index "largest power of two - 1"to size.
After a thread has finished its comparison, it will wait until all the other threads in the block have done the same. This is achieved by using the "__syncthreads()"function.
Note that this function only stops the threads from the same block, so in order to find the max in an array with a size over 2048 we calculate the max in each block and only then do we calculate the maximum among them. In our code we do this by returning an array with the max from each block and then recalling the reduce function with this new array, which now returns an 'array' of size 1.

3 Observations

We ran the program multiple times with different array-sizes, the results are given in table1. Note that for the reduction race conditions where possible for sizes larger than 2048 when we where measuring, but the timing results remain the same (the same amount of operations is executed).
When comparing the different methods, we conclude that for small arrays the CPU is the fastest. This is because modern CPU's are very fast. When the arrays get larger, the atomic becomes the fastest operation. We believe this is because less write operations are necessary which makes it faster because read operations can happen in parallel.Even though the max variable will be very contended, a lot of threads won't need to re-read the max value because they were already smaller to begin with.
For smaller arrays the reduction is also faster than the atomic operation, because reduction does less write operations than the atomic operation. If the array is larger, the more writes the reduction algorithm has to do, while the amount of writes the atomic algorithm has to do is not affected as much.

Array Size	**CPU**	**Atomic Operation**	**Reduction**
512	1	103	13
1024	2	96	15
2048	4	91	17
4096	7	99	20
32768	160	157	100
1048576	2097	462	3726
4194304	8539	1522	20393

Tabel 1: Speed Comparison (in microseconds)

If we where to use less threads, we would logically divide the array in sub-arrays, and use the reduction algorithm in the same way as we do when we use multiple blocks.

When using a 2D array, we can use multiple blocks with the amount of threads equal to the size of a row, we can then find the largest element in each row and save it in the first element of each row (using reduction or atomic operations). Then we can find the largest element of the first column (again using reduction or atomic operations) to find the largest value in the 2D array.